

TEMA 2 – El Proyecto Android

Objetivo del tema

En este tema aprenderás la estructura básica de un proyecto Android, el fichero Manifest y la organización de los recursos del proyecto.

2.1 Estructura general

Hemos realizado un par de ejercicios sencillos para empezar a trabajar con Android Studio. Vamos a ver ahora las bases de lo que realmente es un Proyecto Android, de cómo se organiza el código, la lógica que sigue, etc.

Un Proyecto Android se divide en cuatro piezas fundamentales:

- El fichero **AndroidManifest.xml**: es un fichero que declara parámetros importantes para la ejecución de tu App. Por ejemplo, define su nombre, qué Actividad es la primera en lanzarse, el icono que debe usarse, los permisos de acceso a componentes, etc.
- Las **Actividades**: Gestionan las interfaces y el ciclo de vida. Generalmente, hay una Actividad por pantalla en tu App, aunque no siempre. Las Actividades no contienen toda la lógica de negocio, normalmente se limitan a los eventos de las pantallas.
- Los **Recursos**: Todo lo que hay en la carpeta **/res**, básicamente. Aquí cabe de todo, desde imágenes a textos, estilos, audios, etc. Hay un mecanismo estándar para poder referenciar los Recursos desde las Actividades de forma simple.
- El **Gradle**: La configuración real de un proyecto se hace en estos ficheros de script. Aquí se definen las dependencias, las versiones de la SDK, etc. Android Studio como tal no ‘compila’ o ‘ejecuta’ una aplicación, lanza un script de Gradle que le dice como hacerlo.

En líneas generales, a la hora de comenzar un proyecto nuevo en Android Studio lo primero que se hace es configurar correctamente el Gradle. No es una tarea sencilla. Por el momento, no toques estos ficheros; no es necesario para comenzar a hacer Apps sencillas. A continuación, se prepara el AndroidManifest, especialmente, el tema de los permisos. A partir de aquí, se suele subir una versión del proyecto a un repositorio para comenzar el desarrollo en paralelo de la App.

Muchos programadores a la hora de diseñar Actividades prefieren empezar por los layouts y dejar la lógica para el final; pero cada cuál tiene su propio estilo.

2.2 El Manifest

Todo proyecto Android contiene un archivo principal de configuración llamado **Android-Manifest.xml**. Este archivo establece qué componentes tiene tu App, qué permisos necesita y cómo debe tratarla el sistema.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.buttonandtextview">
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/AppTheme">
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

En su estructura se definen:

- La **identidad** de la App: el identificador único de la App es su **paquete**, dónde se encuentra el código Kotlin de la Actividad.
- **Permisos**: Veremos más adelante que, para que nuestra App pueda salir a internet o usar la cámara, se les tienen que dar permisos explícitamente en el Android-Manifest.xml. Sin ellos, nuestra App no fallará, pero no podrá acceder a esos componentes.
- **Configuración global**: Como el icono o el tema de la App. Todo esto afecta a todas las App, salvo que especifiques otras cosas.
- **Componentes**: Se definen las diferentes Actividades que forman la App. También se definen aquí Servicios, Providers, etc.
- **Intent-filter**: Definen cómo se lanzan los componentes. Por ejemplo, permiten especificar cuál es la Actividad principal que se mostrará al arrancar la App.

Normalmente no es necesario editar el **AndroidManifest.xml** más que una vez: al comenzar un proyecto nuevo. Si posteriormente creamos una Activity nueva, NUNCA hay que crear los ficheros a mano y registrarla a mano en el Manifest.

Importante – No te pases de listo

El IDE de Android ya tiene un montón de herramientas para hacer cambios en el proyecto. Si puedes evitarlo, NUNCA hagas nada a mano. Cambias el nivel de API a mano, y resulta que había que hacerlo en tres sitios, y ya tu App no funciona. Que se encargue el Android Studio.

2.3 Los Recursos

En general, todo aquello que no sea código fuente Kotlin en una App debe estar separado en la carpeta /res. Esto incluye a casi todo: imágenes, sonidos, textos, etc. Esto incluye también a los **layouts** y demás componentes que conforman las interfaces gráficas.

Esto permite una flexibilidad enorme. Si mantenemos por separados los textos en varios idiomas, podemos hacer que nuestra App se muestre por defecto en el idioma del dispositivo. Si ponemos la misma imagen en diferentes resoluciones, se mostrará la que mejor se adecúe a la pantalla del móvil. De la misma forma, es posible diseñar layouts que se adapten a los diferentes tamaños de pantalla, o incluso hacer layouts diferentes para móviles y para tablets.

Todos los **recursos** de nuestra App se almacenan dentro de la carpeta **res** del proyecto. Ya has tenido oportunidad de trastear con ella un poco las prácticas anteriores. Puedes organizar los recursos como prefieras, pero deberías respetar la estructura siguiente:

- **Values:** para cadenas de texto, listas y valores simples.
- **Drawable y mipmap:** imágenes y recursos gráficos.
- **Layout:** interfaces para las actividades.
- **Menu:** definición de menús de opciones.
- **Raw:** otros recursos en cualquier formato que quieras. Puedes poner por ejemplo un PDF aquí, pero corre de tu cuenta, programador, poner el código para acceder a su información.

Cuando se compila la App, estos recursos se incluyen en el paquete APK o App Bundle que se instalará en nuestro dispositivo. Desde nuestro proyecto podemos acceder a los recursos de forma automática mediante **una clase llamada R**. Suponiendo claro que respetes la estructura que te hemos mencionado.

Mediante R podemos acceder a un texto guardado en strings.xml de la subcarpeta **values** desde nuestra activity. Un ejemplo más concreto podría ser cuando le indicamos a una activity el layout que tiene que cargar en su método **onCreate ()**.

```
public void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    setContentView(R.layout.main_layout);  
}
```

Ejercicio 2

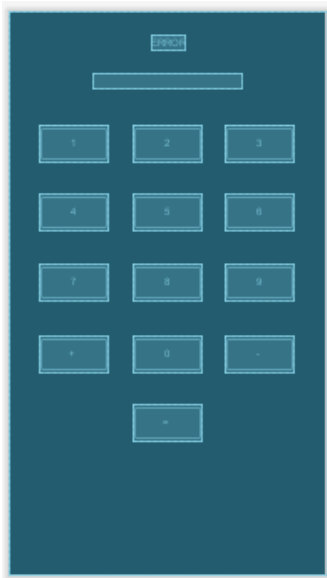
Cambia el icono de una de una App. Android Studio dispone de herramientas para facilitarte la tarea. Si quieres usar una imagen totalmente nueva para el icono, tendrás que incluirla en la carpeta drawable de recursos. Investiga un poco por tu cuenta. Observa que los cambios se registran en el **Android-Manifest.xml**

Ejercicio 3

Crea una calculadora de enteros sencilla. Muestra un teclado con los diez números, el botón de suma, el de resta, y el botón igual. Muestra el resultado de la operación por la pantalla.

La App tendrá 13 **Button** y 2 **TextView**. Mira la imagen para hacerte una idea.

Los **Buttons** estarán marcados con números del 0 al 9. Cada vez que se pulse uno, el número aparecerá en el **TextView** de los resultados. Cuando se pulse el botón de suma o resta, también aparecerá en resultados. Si se pulsa el igual, se borrará el **TextView** y aparecerá el resultado de la operación. Si se pulsa un botón otra vez, podremos volver a operar.



Si hay **errores** como, por ejemplo, cuando convertimos un String en entero, entonces se mostrará un segundo **TextView** inicialmente invisible que contiene la palabra ERROR. La App no puede dejar de funcionar por este motivo bajo ningún concepto. Pulsar cualquier tecla ocultará de nuevo el mensaje de error.

Dispones de una clase **Operations** para que la en la página siguiente. Tiene un solo método público llamado doOperation (String) que es capaz de hacer cosas como “4 + 5” y darte el resultado en String.

Esta práctica la tienes solucionada: **P3 - SimpleCalculatorKotlin.zip**

Class Operations

```
package es.pruebas.simplecalculatorkotlin.operations

@Throws(Exception::class)
fun doOperation(operation: String): String? {
    var ret: String? = null
    var firstNumber = ""
    var lastNumber = ""
    if (isAddition(operation)) {
        firstNumber = operation.substring(0, operation.lastIndexOf("+"))
        lastNumber = operation.substring(operation.lastIndexOf("+") + 1)
        ret =
            "" + add(firstNumber.trim { it <= ' ' }.toInt(),
                lastNumber.trim { it <= ' ' }.toInt())
    } else if (isSubtraction(operation)) {
        firstNumber = operation.substring(0, operation.lastIndexOf("-"))
        lastNumber = operation.substring(operation.lastIndexOf("-") + 1)
        ret = "" + subtract(firstNumber.trim { it <= ' ' }.toInt(),
            lastNumber.trim { it <= ' ' }.toInt())
    }
    return ret
}

private fun isAddition(operation: String): Boolean {
    var ret = false
    val countAdd = operation.length - operation.replace("+", "").length
    val countSubs = operation.length - operation.replace("-", "").length
    if (countAdd == 1 && countSubs == 0) ret = true
    return ret
}

private fun isSubtraction(operation: String): Boolean {
    var ret = false
    val countAdd = operation.length - operation.replace("-", "").length
    val countSubs = operation.length - operation.replace("+", "").length
    if (countAdd == 1 && countSubs == 0) ret = true
    return ret
}

private fun add(op1: Int, op2: Int): Int {
    return op1 + op2
}

private fun subtract(op1: Int, op2: Int): Int {
    return op1 - op2
}
```